



Manual do Programador - Gestão de Sócios

Guia técnico para manutenção da área de Sócios no projeto central.

Ramo	Gestão de Sócios
Destinatários	Programadores e responsáveis técnicos
Repositório	https://github.com/MenteMovimento/central-mente-movimento
Site	https://central-mente-movimento.vercel.app
Atualização	22/06/2026

1. Visão geral da Central

A Central MenteMovimento junta os três ramos num único projeto publicado na Vercel e ligado a um único projeto Supabase. A autenticação é centralizada; os dados continuam separados por tabelas e contexto funcional.

- Repositório oficial: <https://github.com/MenteMovimento/central-mente-movimento>
- Site de produção: <https://central-mente-movimento.vercel.app>
- Branch principal: main.
- Build de produção: `npm run build`.
- Diretório publicado pela Vercel: `public`.
- Não publicar ficheiros `.env`, bases SQLite locais, anexos reais, exports com dados pessoais ou chaves privadas.

2. Variáveis e segurança

- `SUPABASE_URL` e `SUPABASE_ANON_KEY` podem ser usadas pelo frontend.
- `SUPABASE_SERVICE_ROLE_KEY` é secreta e deve ficar apenas em Environment Variables da Vercel ou ambiente local protegido.
- `SUPABASE_SECRET_KEY` é usada pelo backend Python de Utentes quando aplicável.
- Depois de mudar variáveis na Vercel, fazer redeploy.
- Se uma chave secreta for exposta, gerar uma nova no Supabase, atualizar Vercel e revogar a antiga.

3. Fluxo de alteração

- 1 Alterar o código localmente.
- 2 Executar `npm run build` na raiz.
- 3 Testar as páginas afetadas.
- 4 Copiar as alterações para a cópia ligada ao GitHub, se estiver a usar a pasta `.publish`.
- 5 Fazer commit com mensagem clara.
- 6 Fazer push para main.
- 7 Verificar o deployment automático na Vercel.

4. Estrutura do ramo Sócios

- Código principal: `portal/modules/socios/app.js`.
- Interface base: `portal/modules/socios/index.html`.
- Estilos: `portal/modules/socios/styles.css`.
- Ligação Supabase: `portal/modules/socios/central-socios-client.js`.
- Manuais PDF: `portal/modules/socios/docs/`.
- Tabela principal: `public.members`.
- Utilizadores globais: `public.app_users`.

5. Dados e validação

- `members` guarda número de sócio, nome, NIF, contactos, localidade, quota anual, mês pago e data do pagamento.
- O número de sócio deve ser único.
- A importação faz validação antes de gravar; se houver erro, cancelar e corrigir o ficheiro.
- O estado das quotas é calculado no frontend com base no mês pago.

- Alterações relevantes devem ficar no histórico do ramo.

6. Manutenção

- Quando mudar campos de sócios, atualizar app.js, HTML, validação, importação/exportação e manual.
- Se adicionar colunas ao Supabase, criar SQL incremental e testar em projeto de teste.
- Não usar service_role no browser.
- Confirmar que CSV exportado continua a abrir corretamente no Excel/Google Sheets.

Checklist técnica antes de publicar

- 1 Confirmar que as alterações estão na pasta local correta.
- 2 Executar o gerador de manuais quando houver mudança funcional.
- 3 Executar npm run build na raiz do projeto.
- 4 Abrir pelo menos login, dashboard e o ramo alterado.
- 5 Testar navegação entre Sócios, Utentes e Dispositivos.
- 6 Confirmar que tema escuro, idioma e menu mantêm comportamento global.
- 7 Fazer commit e push para main apenas depois dos testes básicos passarem.

Ficheiros principais deste ramo

- portal/modules/socios/index.html - estrutura da página e ligações dos manuais.
- portal/modules/socios/app.js - lógica de listagem, edição, quotas, importação e histórico.
- portal/modules/socios/styles.css - tema visual específico do ramo.
- portal/modules/socios/central-socios-client.js - cliente Supabase usado no browser.
- portal/modules/socios/docs/ - PDFs publicados no botão Manuais.

Dados, tabelas e Storage

- Tabela members guarda dados administrativos dos sócios.
- Tabela app_users é global e não deve ser duplicada por ramo.
- Histórico de alterações deve identificar ação, registo e utilizador.
- Importações devem validar tudo antes de gravar qualquer linha.
- Exports reais devem ficar fora do GitHub.

Regras de compatibilidade

- Não alterar nomes de colunas em produção sem migração SQL e plano de reversão.
- Não remover campos usados por importação, exportação, estatísticas ou histórico.
- Manter a navegação com caminhos absolutos, por exemplo /area/socios/ e não caminhos relativos.
- Manter textos novos nas tabelas de tradução quando o conteúdo aparece na interface.
- Confirmar que o build da Vercel usa os mesmos ficheiros que foram testados localmente.

Diagnóstico de problemas em produção

- Se só a Vercel falhar, verificar logs do deployment e variáveis de ambiente.
- Se Supabase devolver erro de sessão, verificar SUPABASE_SERVICE_ROLE_KEY e permissões.

- Se uma API Python falhar, confirmar importações, caminhos de ficheiros e limites serverless.
- Se um PDF antigo continuar a abrir, limpar cache do browser ou confirmar se o ficheiro foi copiado para public.
- Se houver lentidão, confirmar chamadas repetidas, listas muito grandes e loops de renderização.

Política de dados reais

- Nunca commitar exports reais de sócios, utentes ou dispositivos.
- Nunca commitar anexos reais ou bases SQLite com dados pessoais.
- Usar dados fictícios em testes e prints de suporte.
- Se uma chave ou ficheiro sensível for publicado por engano, revogar, substituir e registar o incidente.